

Studio sperimentale di una coda MPMC bounded con prenotazione FAA in MoStream

Relazione tecnica preliminare per attività di tesi

Progetto MoStream - 21 luglio 2026

Questa relazione documenta il confronto tra la coda MPMC bounded originale di MoStream, basata su Compare-And-Swap (CAS), e una variante sperimentale che usa Fetch-And-Add (FAA) per assegnare ticket nelle operazioni bloccanti. Sono presentati struttura degli algoritmi, metodologia, risultati, limiti e criteri di applicabilità. Nei 18 scenari provati la variante FAA ha ottenuto throughput superiore e non ha perso o duplicato messaggi. Il vantaggio cresce con la contesa, ma la prenotazione non annullabile introduce un rischio di head-of-line blocking che impedisce di considerare FAA una sostituzione universalmente migliore senza ulteriori test di progresso e integrazione.

Obiettivo e terminologia

L'obiettivo è verificare sperimentalmente se la prenotazione FAA possa essere applicata alle code concorrenti di MoStream e quali compromessi comporti rispetto all'implementazione originale.

FAA e MPMC non sono categorie contrapposte. MPMC significa Multiple Producer, Multiple Consumer e descrive la semantica della coda. Entrambe le implementazioni studiate sono code MPMC bounded. Nel seguito si usano le denominazioni:

- CAS-MPMC: implementazione originale basata sull'algoritmo bounded MPMC di Dmitry Vyukov, con prenotazione tramite `compare_exchange`;
- FAA-MPMC: variante sperimentale dello stesso ring buffer, con `fetch_add` nelle operazioni bloccanti push e pop.

La domanda sperimentale è quindi: una MPMC bounded con ticket FAA è più efficiente e sufficientemente robusta per i workload di MoStream rispetto alla MPMC bounded con CAS?

Struttura comune delle implementazioni

Le due code condividono i seguenti elementi:

- ring buffer di capacità fissa, necessariamente potenza di due e almeno pari a due;
- indice dello slot calcolato applicando una mask al contatore monotono;
- sequence number atomico per ogni cella, usato per distinguere generazioni e stato dello slot;
- contatori `enqueue_pos` e `dequeue_pos` separati e padded a 64 byte per ridurre il false sharing;
- letture ACQUIRE e pubblicazioni RELEASE per rendere visibili i dati tra producer e consumer;
- payload memorizzato come Optional e trasferito mediante move.

Per il ticket `p` di un producer, sequence uguale a `p` indica uno slot libero; sequence uguale a `p+1` indica un elemento pubblicato. Dopo il consumo del ticket `c`, il consumer imposta sequence a `c+capacity`, rendendo la cella disponibile nel giro successivo del ring.

Differenza algoritmica principale

Prenotazione CAS

Nella coda originale un thread legge la posizione corrente, controlla la sequence della cella e tenta `compare_exchange(position, position+1)`. Più thread possono osservare la stessa posizione: uno vince, mentre gli altri falliscono il CAS e ripetono il ciclo. All'aumentare della contesa crescono i retry e il traffico di coerenza sulla cache line del contatore.

Prenotazione FAA

Nella variante sperimentale push e pop bloccanti eseguono un solo `fetch_add(1)`. Ogni thread riceve immediatamente un ticket distinto e attende che la cella associata raggiunga la sequence prevista. Si evitano così le collisioni CAS sul contatore centrale.

Il ticket ottenuto con FAA non può essere annullato. Se lo slot non è pronto, il thread deve aspettarlo. L'attesa esegue `spin` e, dopo 1024 osservazioni senza successo, `sleep(0.0)` per cedere occasionalmente la CPU allo scheduler.

Operazioni non bloccanti

`try_push` e `try_pop` continuano a usare CAS anche nella variante FAA. Usare `fetch_add` in queste operazioni lascerebbe un buco permanente se il tentativo non potesse completarsi e il ticket venisse abbandonato. Di conseguenza, i risultati di questo studio riguardano principalmente push e pop bloccanti.

Garanzie di progresso e rischio principale

La FAA-MPMC implementata non è la coda wait-free completa di Yang e Mellor-Crummey. Applica soltanto l'idea di assegnare posizioni distinte tramite FAA. I metodi bloccanti possono attendere indefinitamente lo slot del proprio ticket.

Lo scenario critico è il seguente:

1. un thread ottiene il ticket N;
2. il thread viene descheduled, sospeso o termina prima della pubblicazione;
3. il ticket N non viene completato;
4. consumer o producer successivi possono restare in attesa dell'avanzamento associato a quel ticket.

Questo fenomeno è head-of-line blocking. FAA riduce il costo della contesa quando tutti i partecipanti progrediscono, ma aumenta la dipendenza dal progresso del thread proprietario del ticket. Pertanto un miglior throughput non implica automaticamente una garanzia di progresso migliore.

Problemi e differenze osservati nel codice

- Il backoff CAS originale contiene un ciclo vuoto che il compilatore può eliminare, quindi non costituisce necessariamente un backoff effettivo.
- La FAA-MPMC usa `spin` seguito da `sleep(0.0)`; il diverso backoff può influenzare il confronto e dovrà essere isolato in un esperimento dedicato.
- Il move constructor originale azzerava `enqueue_pos` e `dequeue_pos`. Dopo lo spostamento di una coda usata, i contatori possono diventare incoerenti con le sequence memorizzate. La variante FAA conserva le posizioni.
- `estimated_len` della CAS-MPMC chiamava `load` sul wrapper padded invece che sul campo `atomicVal`, impedendo la compilazione del modulo. La chiamata è stata

corretta per poter costruire il benchmark.

- Le due code gestiscono in modo diverso una capacità non valida: l'originale solleva Error, mentre la variante sperimentale termina il processo con exit(1).
- FAA_Queue.mojo definisce ancora il tipo con il nome MPMCQueue. Per una futura integrazione conviene adottare nomi o alias non ambigui.
- Al momento communicator.mojo e scheduler.mojo importano ancora MoStream.MPMC_queue; FAA_Queue non è collegata al runtime principale.

Metodologia sperimentale

Il benchmark usa Int come payload. Ogni producer inserisce un intervallo disgiunto di interi non negativi. L'ultimo producer che termina inserisce un sentinel -1 per ogni consumer. Ogni consumer mantiene localmente conteggio e checksum durante la regione temporizzata. Dopo il join i risultati locali vengono aggregati e confrontati con:

```
conteggio atteso = producer x messaggi_per_producer
checksum atteso  = N x (N - 1) / 2
```

L'uso di contatori locali evita di introdurre un'operazione atomica condivisa per ogni messaggio. Ogni esecuzione è valida soltanto se conteggio e checksum coincidono, rilevando elementi persi o duplicati. Il benchmark alterna l'ordine CAS/FAA tra le ripetizioni per limitare il bias dovuto all'ordine di esecuzione.

La matrice comprende topologie bilanciate 1P-1C, 2P-2C, 4P-4C e 8P-8C, più i casi sbilanciati 1P-8C e 8P-1C. Le capacità sono 16, 1024 e 65536. Sono stati usati 250000 messaggi per producer e 10 ripetizioni per configurazione. Il throughput è espresso in milioni di messaggi al secondo. Sono calcolati media, deviazione standard campionaria e intervallo di confidenza bilaterale al 95% con Student-t.

Ambiente di esecuzione

```
tab( );
Componente Valore
CPU Intel Xeon Gold 5512U
Topologia 1 socket, 28 core, 2 thread/core, 56 CPU logiche
Architettura x86_64, 1 nodo NUMA
Sistema Linux 5.15.0-168-generic
Mojo 1.0.0b1, revisione a9591de6
```

I risultati descrivono questa specifica esecuzione e non sono automaticamente generalizzabili ad altre microarchitetture, sistemi operativi o versioni del compilatore.

Risultati

La tabella riporta il throughput medio e lo speedup medio FAA/CAS. Tutte le 180 coppie queue-configurazione-ripetizione hanno superato il controllo di correttezza.

```
tab( ), box;
Topologia Capacità CAS Mmsg/s FAA Mmsg/s Speedup
1P-1C 16 7.087 9.127 1.31x
1P-1C 1024 5.416 7.730 1.43x
1P-1C 65536 5.520 7.175 1.30x
2P-2C 16 4.074 6.520 1.60x
2P-2C 1024 4.059 6.378 1.57x
2P-2C 65536 4.625 7.400 1.60x
4P-4C 16 3.254 8.535 2.63x
4P-4C 1024 3.374 8.214 2.43x
4P-4C 65536 3.650 9.016 2.47x
```

8P-8C	16	2.535	9.776	3.86x
8P-8C	1024	2.712	8.868	3.27x
8P-8C	65536	2.652	9.565	3.61x
1P-8C	16	3.408	4.642	1.37x
1P-8C	1024	3.461	4.240	1.23x
1P-8C	65536	3.748	4.195	1.12x
8P-1C	16	3.390	4.962	1.46x
8P-1C	1024	3.845	5.059	1.32x
8P-1C	65536	3.881	5.460	1.41x

Gli intervalli di confidenza completi e le deviazioni standard sono conservati nel file results.csv accanto al benchmark. Alcuni esempi di speedup con intervallo di confidenza al 95% sono: 1P-1C/capacità 1024, 1.43 +/- 0.07x; 4P-4C/capacità 1024, 2.43 +/- 0.03x; 8P-8C/capacità 16, 3.86 +/- 0.03x; 1P-8C/capacità 65536, 1.12 +/- 0.04x.

Interpretazione

Scalabilità sotto contesa

FAA è risultata più veloce in tutte le 18 configurazioni. Il vantaggio cresce nelle topologie bilanciate: circa 1.3-1.4x con 1P-1C, circa 1.6x con 2P-2C, circa 2.4-2.6x con 4P-4C e circa 3.3-3.9x con 8P-8C. Questo andamento è coerente con l'ipotesi che FAA elimini i retry CAS sul contatore condiviso.

Workload sbilanciati

Con 1P-8C e 8P-1C il vantaggio scende a circa 1.1-1.5x. Quando uno dei due lati ha un solo partecipante, la contesa su uno dei contatori è ridotta e FAA ha meno collisioni da eliminare. Il risultato suggerisce che la scelta della coda dipenda anche dalla topologia del canale.

Capacità

Non emerge una capacità universalmente ottimale. L'effetto della capacità è secondario rispetto al numero di producer e consumer e può dipendere da cache, pattern di scheduling e pressione del ring. Sono necessari test più lunghi e controllati prima di attribuire significato causale alle differenze tra 16, 1024 e 65536.

Correttezza osservata

Nessuna esecuzione ha mostrato perdita o duplicazione di elementi. Questo è un risultato positivo, ma non costituisce una prova formale di correttezza e non copre ogni possibile interleaving, overflow UInt64 o failure mode.

Applicabilità a MoStream

Communicator delle pipeline

FAA-MPMC è una candidata forte per i communicator quando i worker sono stabili, push/pop sono bloccanti e la contesa è medio-alta. I risultati mostrano un beneficio crescente proprio nelle topologie con più producer e consumer.

Queue dello scheduler

L'adozione nello scheduler richiede maggiore cautela. Un worker che ottiene un ticket e viene sospeso può trattenere il progresso di altri worker. Poiché lo scheduler governa il progresso dell'intero sistema, starvation o head-of-line blocking hanno conseguenze più gravi che in un semplice canale.

Inoltre lo scheduler usa frequentemente `try_push/try_pop`, per i quali la variante conserva CAS e potrebbe non offrire lo stesso vantaggio misurato.

Strategia consigliata

Non sostituire immediatamente tutte le code. Rendere l'implementazione selezionabile e procedere per fasi:

1. integrare FAA nei soli communicator;
2. misurare pipeline reali end-to-end;
3. aggiungere stress test con worker sospesi;
4. valutare separatamente le queue dello scheduler;
5. scegliere eventualmente una politica ibrida in base al tipo di coda.

Limiti dello studio

- Una sola macchina, una sola CPU e una sola versione di Mojo.
- Dieci ripetizioni sono utili per una valutazione preliminare, ma una campagna definitiva dovrebbe usarne almeno 30 e registrare lo stato della macchina.
- Assenza di pinning esplicito dei thread e mancato controllo di frequenza CPU, turbo, carico esterno e temperatura.
- Misurato throughput aggregato, non latenza mediana, p95, p99 o worst case.
- Payload limitato a Int; payload grandi o con move costoso possono modificare il peso relativo della sincronizzazione.
- Benchmark sintetico; non include il lavoro degli stage e le interazioni reali del runtime MoStream.
- Nessuna misura hardware di CAS falliti, cache miss, context switch, consumo CPU o energia.
- Il backoff differisce tra implementazioni e potrebbe spiegare una parte del vantaggio.
- Non sono ancora stati eseguiti test mirati di sospensione dopo la prenotazione FAA, crash del ticket owner o oversubscription estrema.

Esperimenti futuri prioritari

1. Instrumentare i retry CAS e il numero di spin/yield FAA.
2. Misurare latenza p50, p95, p99 e massima con campionamento a basso overhead.
3. Sospendere intenzionalmente un thread subito dopo `fetch_add` e quantificare head-of-line blocking, starvation e tempo di recupero.
4. Confrontare backoff equivalenti: spin puro, pause instruction, exponential backoff e yield periodico.
5. Provare payload da 8, 64, 256 byte e oggetti con move non banale.
6. Eseguire almeno 30 ripetizioni, randomizzare l'ordine e riportare intervalli di confidenza e dimensione dell'effetto.
7. Ripetere su architetture diverse, almeno x86-64 e ARM64, e con differenti topologie NUMA.
8. Integrare selettivamente FAA nei communicator e confrontare pipeline MoStream reali, incluse configurazioni con stage sbilanciati.
9. Analizzare formalmente linearizzazione, memory ordering, wrap-around e garanzie di progresso.

Conclusione

I dati supportano l'ipotesi che la prenotazione FAA sia una scelta efficace per ridurre la contesa nelle operazioni bloccanti di una coda MPMC bounded. In tutte le configurazioni esaminate FAA ha migliorato il throughput, con un vantaggio che raggiunge circa 3.9x sotto elevata contesa bilanciata. I controlli di

conteggio e checksum non hanno rilevato errori.

La conclusione non deve però essere formulata come superiorità universale. FAA scambia i retry CAS con ticket non annullabili e attesa dello slot assegnato.

Il possibile head-of-line blocking è particolarmente rilevante nello scheduler.

La valutazione più corretta è quindi: FAA-MPMC è una candidata molto promettente per i communicator di MoStream e per workload ad alta contesa con worker che progrediscono regolarmente; l'adozione nello scheduler o come default globale richiede ulteriori prove di latenza, progresso, failure behavior e prestazioni end-to-end.

Materiale riproducibile

I file prodotti nel repository sono:

- Benchmarks/QueueBenchmark/queue_benchmark.mojo: benchmark concorrente e controllo di correttezza;
- Benchmarks/QueueBenchmark/run_suite.py: compilazione, matrice sperimentale, statistiche ed esportazione;
- Benchmarks/QueueBenchmark/results.csv: risultati completi;
- MoStream/FAA_Queue.mojo: implementazione sperimentale.

La suite è riproducibile dalla radice del repository con:

```
python3 Benchmarks/QueueBenchmark/run_suite.py \  
250000 30 Benchmarks/QueueBenchmark/results.csv
```